Fine-Tuned GPT-2 Language Models for Hindi and Hinglish Next-Word Prediction and Sentence Completion

Samir (blackirron)
IIT (ISM) Dhanbad
`github.com/blackirron`
CS224N Final Project (Custom Project)

Abstract

Predictive text and sentence completion are mature technologies for English, but far less developed for Hindi and for Hinglish, the code-mixed Hindi-English register that dominates everyday digital communication (WhatsApp, chat, social media) among Hindi speakers. Standard pretrained language models struggle with this setting because their tokenizers are built for a single script and register: a Devanagari-only tokenizer cannot represent romanized text at all, and a broadcast-register corpus (e.g. Twitter) does not reflect the two-person conversational register of a chat application. We build and evaluate two complementary systems: (1) a Devanagari Hindi next-word predictor and sentence completer, built by fine-tuning `surajp/gpt2-hindi` on conversational Hindi subtitle data, and (2) a Hinglish (romanized, code-mixed) system, built by fine-tuning `l3cube-pune/hing-gpt` on synthetic two-person Hinglish dialogue. Fine-tuning reduces held-out perplexity and produces a qualitative shift from formal/broadcast register toward conversational register in both systems. We additionally document and address several tokenizer-level engineering pitfalls – out-of-range special token IDs, missing pad/eos tokens, and a complete absence of word-boundary marking in the Hinglish tokenizer – that are not specific to our models but are likely to recur whenever practitioners fine-tune community-released, non-mainstream tokenizers.

1 Introduction

Predictive keyboards and sentence-completion systems are a core, everyday application of language modeling, but the vast majority of production systems are built for English. For the roughly 600 million speakers of Hindi and Hindi-English code-mixed "Hinglish," predictive text support is comparatively immature, despite Hinglish being the dominant register of everyday messaging in India.

This gap is not simply a data availability problem – it is also a tokenization and register problem. A model whose tokenizer was built exclusively on Devanagari script text has no meaningful representation of romanized Hindi at all: a word like *main* ("I") is not merely rare in such a vocabulary, it is entirely absent, forcing degenerate byte- or character-level fallbacks. Conversely, a model trained on romanized Hindi-English text scraped from a broadcast platform like Twitter learns a public, one-to-many register quite different from the private, two-person, question-and-response register of a chat application such as WhatsApp.

We address both problems directly by building two separate, purpose-fit pipelines rather than one pipeline stretched across both scripts:

- **A Devanagari Hindi system**, fine-tuned on conversational (subtitle) Hindi text, for standard-script next-word prediction and sentence completion.
- **A Hinglish system**, fine-tuned on synthetic two-person conversational Hinglish dialogue, for romanized, code-mixed prediction and completion – the register actually used in WhatsApp-style chat.

For both systems we start from small (88M–124M parameter) GPT-2-architecture models that were already pretrained on the relevant script/register combination, and fine-tune them further on a conversationally-matched corpus. This lets us run the entire

pipeline – fine-tuning, evaluation, and inference – on a single free-tier Colab GPU, which we view as a feature rather than a limitation: it demonstrates that meaningful, measurable improvements in a low-resource, code-mixed setting do not require large-scale compute.

Our main contributions are: (1) two working, evaluated next-word-prediction and sentence-completion pipelines for Hindi and Hinglish; (2) a quantitative and qualitative comparison of base vs. fine-tuned models in both settings; and (3) a documented account of tokenizer-level failure modes – particularly a complete absence of word-boundary information in one of the two tokenizers we used – that we believe generalize beyond our specific models.

2 Related Work

Radford et al. (2019) introduced the GPT-2 architecture and byte-level BPE tokenization scheme that underlies both models we fine-tune. Authors (2024) present HindiLLM, a from-scratch pretrained Hindi language model family, and explicitly identify the combination of Hindi, romanized Hindi, and English data as future work – the exact gap our two-pipeline approach is designed to fill by treating the Devanagari and romanized settings as separate systems rather than attempting a single joint model.

Nayak and Joshi (2022) introduce L3Cube-HingCorpus, a large-scale (52.93M sentence) real Hindi-English code-mixed Twitter corpus, along with HingBERT, HingRoBERTa, and HingGPT – the base generative model we fine-tune for our Hinglish pipeline. Their work demonstrates that models pretrained natively on romanized code-mixed text substantially outperform models adapted post-hoc from monolingual tokenizers, which directly motivated our choice to use HingGPT rather than attempt to adapt our Devanagari model to romanized input (an approach we empirically confirm fails in Section 5).

For fine-tuning data, our Devanagari pipeline draws on OpenSubtitles (Lison and Tiedemann, 2016), a large parallel/monolingual corpus of movie and TV subtitle dialogue, which we use for its informal, conversational register rather than its translation alignment. Our Hinglish pipeline draws on a synthetically-generated two-person Hinglish conversation dataset, in the spirit of recent work on sample-efficient Hinglish conversational modeling (Singh et al., 2025), which similarly uses synthetic dialogue generation to address the scarcity of naturally-occurring, privacy-safe conversational (as opposed to broadcast) code-mixed data.

3 Approach

3.1 Base models

For the Devanagari pipeline, we use `surajp/gpt2-hindi`, an 88M-parameter GPT-2 model pretrained from scratch on Hindi text using a causal language modeling objective, with a byte-level BPE tokenizer (vocabulary size 50,257) native to Devanagari script.

For the Hinglish pipeline, we use `l3cube-pune/hing-gpt` (Nayak and Joshi, 2022), a 12-layer GPT-2 model pretrained from scratch on L3Cube-HingCorpus. Its tokenizer has a WordPiece-style vocabulary of size 30,522 and, notably, ships without a defined padding or end-of-sequence token.

3.2 Fine-tuning

Both models are fine-tuned with the standard causal language modeling objective (next-token cross-entropy) using the Hugging Face `Trainer` API. Training text is joined with an end-of-sequence separator and chunked into fixed-length blocks of 128 tokens; we use a 95/5 train/validation split, batch size 16, learning rate 5×10^{-5} with 100 warmup steps, weight decay 0.01, and mixed-precision (fp16) training for 3 epochs on a single T4 GPU.

Two tokenizer-level issues required explicit handling before fine-tuning could run correctly. First, we found that `surajp/gpt2-hindi`'s shipped configuration referenced a `pad_token_id` one index past its own embedding matrix – an out-of-bounds reference that manifests as a CUDA device-side assertion failure the moment the token is embedded. Second, because `l3cube-pune/hing-gpt`'s tokenizer defines no end-of-sequence token at all, we add one as a new special token and correspondingly resize the model's embedding matrix (via

`resize_token_embeddings`) to accommodate it; this in turn requires manually correcting stale `bos_token_id`/`eos_token_id` values left over in the model configuration from before the resize, which otherwise point past the (now smaller) embedding matrix and reproduce the same class of out-of-bounds failure. We view these as representative, not idiosyncratic: any practitioner fine-tuning a community-released checkpoint with a modified or incomplete tokenizer configuration should expect to encounter this failure mode.

3.3 Next-word prediction

Next-word prediction requires returning whole-word candidates, but both tokenizers operate at the subword level, and the two require materially different handling.

For the Devanagari (BPE) tokenizer, a token that begins a new word is systematically prefixed with a leading space in its decoded form. We exploit this: for each of the top-k first-token candidates (by softmax probability), we greedily extend the candidate token-by-token, using the *next* predicted token's leading-space status as the stopping signal, until a word boundary is reached or a step budget is exhausted. We additionally add a Devanagari-specific well-formedness check: a candidate word may not end on a bare virama (the conjunct-joining diacritic ◌्), since this is never grammatically valid, and we force one further extension step in that case.

For the Hinglish (WordPiece-style) tokenizer, we found this approach does not transfer. Critically, we verified that this tokenizer's encoding is *lossy with respect to word boundaries*: the strings *whatsappkarna* (one word) and *whatsapp karna* (two words) encode to *identical* token ID sequences, and the tokenizer's raw subword pieces carry no boundary marker of any kind (no leading space, no `##` continuation prefix). There is therefore no reliable signal in the token stream from which to reconstruct word boundaries algorithmically. We instead rely on an empirical property of this specific vocabulary: virtually all common Hindi, Hinglish, and English words tokenize to a single token (verified over a sample of common conversational vocabulary), so we treat each top-k candidate token directly as a complete candidate word, at the cost of occasionally returning a fragment for genuinely out-of-vocabulary words.

3.4 Sentence completion

Sentence completion is capped at a target number of sentences (3, by default) rather than a fixed token budget, since we found a fixed budget produces completions of inconsistent, often unnatural length. For the Devanagari model we implement a manual token-by-token sampling loop (top-p = 0.95, temperature 0.8) so that each generated token can be inspected for sentence-ending punctuation (`.`, `?`, `!`, and the Devanagari *danda* `।`) as it is produced. For the Hinglish model we instead use Hugging Face's `generate()` with repetition penalty (1.2) and a no-repeat 2-gram constraint – necessary because we observed the un-penalized base model enter degenerate repetition loops on some prompts – and regenerate with a growing token budget (20, 40, 60, …) until the newly generated text (explicitly excluding punctuation already present in the prompt, a bug we identify and fix in Section 6) contains enough sentence-ending punctuation.

4 Experiments

4.1 Data

Devanagari corpus. We extract the Hindi side of the OpenSubtitles English-Hindi parallel corpus (Lison and Tiedemann, 2016), obtained directly from OPUS, and clean it (length filtering, artifact stripping, deduplication) to 76,750 conversational Hindi sentences (∼1.6M tokens after tokenization).

Hinglish corpus. We sample 100,000 conversation turns from `Hinglish-Everyday-Conversations-1M`, a synthetically-generated (GPT-4o-mini) dataset of two-person Hinglish dialogue on everyday topics, cleaning and deduplicating to 199,583 lines (∼4.08M tokens after tokenization) – chosen specifically for its person-to-person chat register, in contrast to HingCorpus's broadcast-register Twitter data.

4.2 Evaluation method

We report held-out perplexity (exponentiated cross-entropy loss on a 5% validation split) as our primary quantitative metric. Because per-

plexity alone does not capture register or fluency, we complement it with qualitative comparison of base vs. fine-tuned model completions on a fixed, shared set of test prompts for each language setting.

4.3 Experimental details

Both models are fine-tuned for 3 epochs with the hyperparameters described in Section 3.2 on a single Google Colab T4 GPU. The Devanagari run (31,900 total blocks would be produced by the raw token count, though only ∼12,600 fit our 128-token block size at this corpus size) took under 10 minutes; the larger Hinglish run (31,900 training blocks) took approximately 22 minutes.

4.4 Results

Table 1 reports held-out perplexity before and after fine-tuning is not directly comparable across the two systems (different tokenizers, vocabularies, and corpora), so we report each system's fine-tuned perplexity alongside qualitative base-vs-fine-tuned comparison instead of a single combined leaderboard number.

System	Val. loss	Perplexity
Devanagari (Hindi)	1.891	6.62
Hinglish (code-mixed)	2.379	10.79

Table 1: Final held-out results after 3 epochs of fine-tuning.

The Hinglish system's higher perplexity is expected rather than concerning: code-mixed text is inherently higher-entropy than monolingual text, since it combines two overlapping vocabularies with inconsistent, non-standardized spelling (e.g. *kyu/kyun/kyon* for the same word) and code-switch points that are genuinely harder to predict than remaining in a single language throughout.

Qualitatively, fine-tuning produces a clear, consistent register shift in both systems. The base Devanagari model, when prompted with तुम क्या ("you what"), produces a formal, news-register continuation (तुम क्या है? फोटोग्राफी और सिद्धांत में...); the fine-tuned model produces a direct, conversational continuation (तुम क्या कर रहे हो? – "what are you doing?"). Similarly, the base Hinglish model, prompted with *yeh movie*, drifts toward broadcast-register, occasionally politically-adjacent content; the fine-tuned model produces on-topic, conversational, question-and-response dialogue characteristic of the fine-tuning corpus.

5 Analysis

Overfitting signal. The Devanagari model's validation loss decreases from 1.930 (epoch 1) to 1.896 (epoch 2) but only marginally further to 1.891 (epoch 3), while training loss continues to decrease steadily across all three epochs (1.969 → 1.865 → 1.808). This divergence is an early, mild overfitting signal; we did not train further, judging that a fourth epoch was likely to widen the train/validation gap without a meaningful validation improvement. By contrast, the Hinglish model's validation loss was still decreasing at a similar rate to its training loss through epoch 3 (2.517 → 2.413 → 2.379), suggesting this run had not yet reached its overfitting point and could plausibly benefit from additional training epochs – a natural direction for future work.

Tokenizer-boundary failure modes. The Devanagari next-word predictor occasionally truncates longer, multi-token words before they are grammatically complete (e.g. producing संयुक् instead of संयुक्त, "united"). We partially addressed this with a Devanagari-specific well-formedness check (a word cannot validly end on a bare virama), but a residual failure mode remains where the underlying greedy per-step decoding converges on a locally-plausible but grammatically incomplete stopping point (e.g. मिलेंग instead of मिलेंगे, "will meet"); we believe this specific residual case would require beam search over candidate extensions rather than greedy decoding to fully resolve.

The Hinglish predictor exhibits a structurally different failure mode, stemming from the tokenizer's complete lack of boundary marking: on genuinely out-of-vocabulary words, `decode()` unconditionally inserts a space between subword pieces regardless of whether the original text had one, occasionally splitting a single word into two apparent words in the output (e.g. *Lion* → *L ion* in one observed sentence completion, where our fine-tuning corpus evidently did not contain enough instances of "Lion" as a single token).

A prompt-echo bug and its fix. Dur-

ing testing we discovered that the Hinglish `complete_sentence` function, when given a prompt that itself already contained three or more sentences of punctuation (for instance, a user pasting a prior completion back in as a new prompt), would return the input completely unchanged, having generated no new text. The root cause was that our sentence-counting stopping condition scanned the *entire* decoded output – prompt included – for sentence-ending punctuation, rather than only the newly generated continuation; a prompt that already satisfied the stopping condition on its own caused the function to terminate before any new generation was meaningfully used. We fixed this by decoding the prompt separately and restricting the punctuation count and cut-point search to the substring following the prompt. We note that the equivalent Devanagari implementation, which uses a manual token-by-token generation loop, was not affected by this bug class, since it only ever inspects newly generated tokens as they are produced rather than re-scanning the full decoded text.

6 Conclusion

We built and evaluated two fine-tuned GPT-2-based systems for Hindi and Hinglish next-word prediction and sentence completion, demonstrating that small (88M–124M parameter), free-tier-GPU-trainable models can achieve a clear, measurable register shift toward conversational text through targeted fine-tuning on register-matched data. Our main limitation is that word-level prediction quality is bounded by tokenizer design in ways that differ qualitatively between the two systems we built: BPE's leading-space convention allows algorithmic word-boundary reconstruction (imperfectly, as our residual truncation cases show), while the Hinglish tokenizer's complete absence of boundary marking rules this out entirely, forcing a coarser single-token-as-word approximation. Future work includes: extending the Hinglish model's fine-tuning past 3 epochs, given it had not yet plateaued; replacing greedy per-step word extension with beam search for the Devanagari predictor; and, as originally motivated, integrating the Hinglish system into an actual WhatsApp-style keyboard suggestion interface, which was the real-world use case motivating this project.

References

Various Authors. 2024. HindiLLM: Large language model for hindi. *arXiv preprint arXiv:2412.20357*.

Pierre Lison and Jörg Tiedemann. 2016. OpenSubtitles2016: Extracting large parallel corpora from movie and TV subtitles. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC'16)*, pages 923–929.

Ravindra Nayak and Raviraj Joshi. 2022. L3Cube-HingCorpus and HingBERT: A code mixed hindi-english dataset and bert language models. In *Proceedings of the WILDRE-6 Workshop within the 13th Language Resources and Evaluation Conference*.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Sakshi Singh, Abhinav Prakash, Aakriti Shah, Chaitanya Sachdeva, and Sanjana Dumpala. 2025. Sample-efficient language model for hinglish conversational AI. *arXiv preprint arXiv:2504.19070*.